Alejandro Palacio Caro

Instructor: Dr. Ming Yu

Course: Computer Architecture

Project 3: Single Cycle Datapath

04/20/2026

1. Introduction

This project focuses on designing and implementing a single-cycle datapath processor based on the LEGv8 (ARMv8) architecture using Logisim. The goal is to understand how instructions are executed in a CPU by constructing the datapath and control logic required to support a subset of instructions.

The processor operates using a single-cycle execution model, where each instruction is completed in one clock cycle. The design includes key components such as the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), Data Memory, multiplexers, and supporting logic blocks.

The processor is designed to support arithmetic, logic, memory, and branch instructions, demonstrating how data flows through the datapath and how control signals coordinate the operation of different components.

2. Datapath Overview

The datapath is the main structure of the processor and connects all functional units together. It is responsible for fetching instructions, decoding them, executing operations, accessing memory, and writing results back to registers.

The datapath consists of several interconnected modules:

- Program Counter (PC)
- Instruction Memory
- Control Unit
- Register File
- ALU
- Data Memory
- Multiplexers (MUXes)
- Sign Extend Unit
- Branch Logic and Adders

Each component plays a specific role in the execution of instructions, and the control signals determine how data flows through the datapath.

3. Implemented Instructions

The processor supports the following LEGv8 instructions:

- ADD – Performs addition between registers
- SUB – Performs subtraction between registers
- AND – Bitwise AND operation
- ORR – Bitwise OR operation
- LDUR – Loads data from memory into a register
- STUR – Stores data from a register into memory
- CBZ – Conditional branch if zero
- B – Unconditional branch

4. Control Signal Design

The Control Unit generates control signals based on the opcode of each instruction. A truth table was created to map each instruction to the corresponding control signals. These signals determine how the datapath operates, including selecting inputs, enabling writes, and controlling memory access.

5. Control Unit Table

Instruction	Opcode [31–21]	Reg2 Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
ADD	10001011000	0	0	0	1	0	0	0	1	0
SUB	11001011000	0	0	0	1	0	0	0	1	0
AND	10001010000	0	0	0	1	0	0	0	1	0
ORR	10101010000	0	0	0	1	0	0	0	1	0
LDUR	11111000010	X	1	1	1	1	0	0	0	0
STUR	11111000000	1	1	X	0	0	1	0	0	0
CBZ	10110100XXX	1	0	X	0	0	0	1	0	1
B	000101XXXXX	X	X	X	0	0	0	1	0	0

6. Test Program

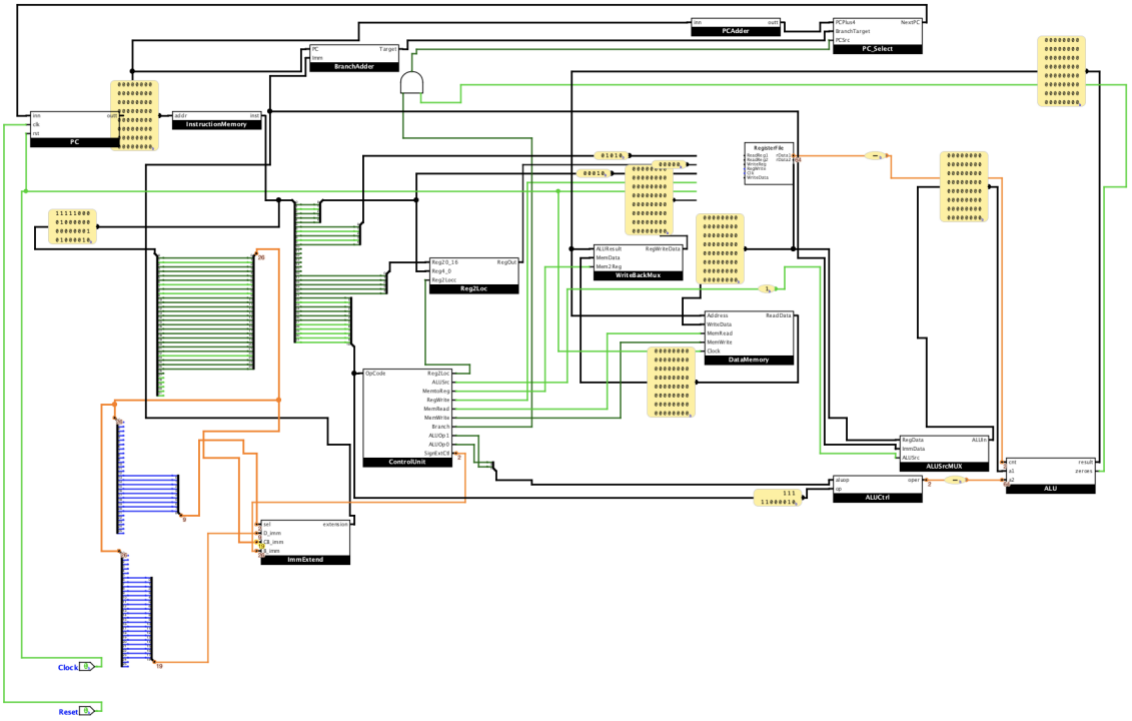
he following instructions were used to test the functionality of the processor:

Line	Instruction	Hex
1	LDUR r2, [r10]	0xF8400142
2	LDUR r3, [r10, #1]	0xF8401143
3	SUB r4, r3, r2	0xCB020064
4	ADD r5, r3, r2	0x8B020065
5	CBZ r1, #2	0xB4000041
6	CBZ r0, #2	0xB4000040
7	LDUR r2, [r10]	0xF8400142
8	ORR r6, r2, r3	0xAA030046
9	AND r7, r2, r3	0x8A030047
10	STUR r4, [r7, #1]	0xF80010E4
11	B #2	0x14000003
12	LDUR r3, [r10, #1]	0xF8401143
13	ADD r8, r0, r1	0x8B010008

7. Module Descriptions

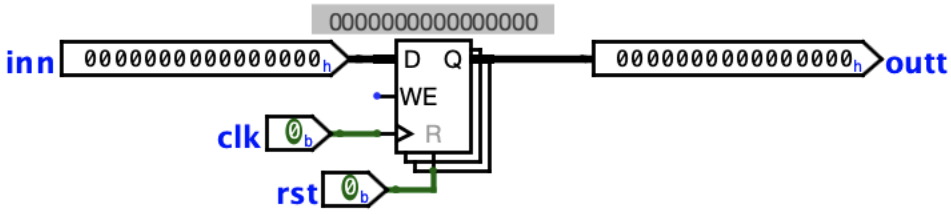
Data Path

The datapath integrates all major components of the processor, including the Program Counter, Instruction Memory, Control Unit, Register File, ALU, Data Memory, and multiplexers. It defines the flow of data during instruction execution. The datapath supports instruction fetch, decode, execute, memory access, and write-back stages within a single clock cycle. Control signals generated by the Control Unit guide the movement of data through the system, enabling correct execution of arithmetic, memory, and branch instructions.



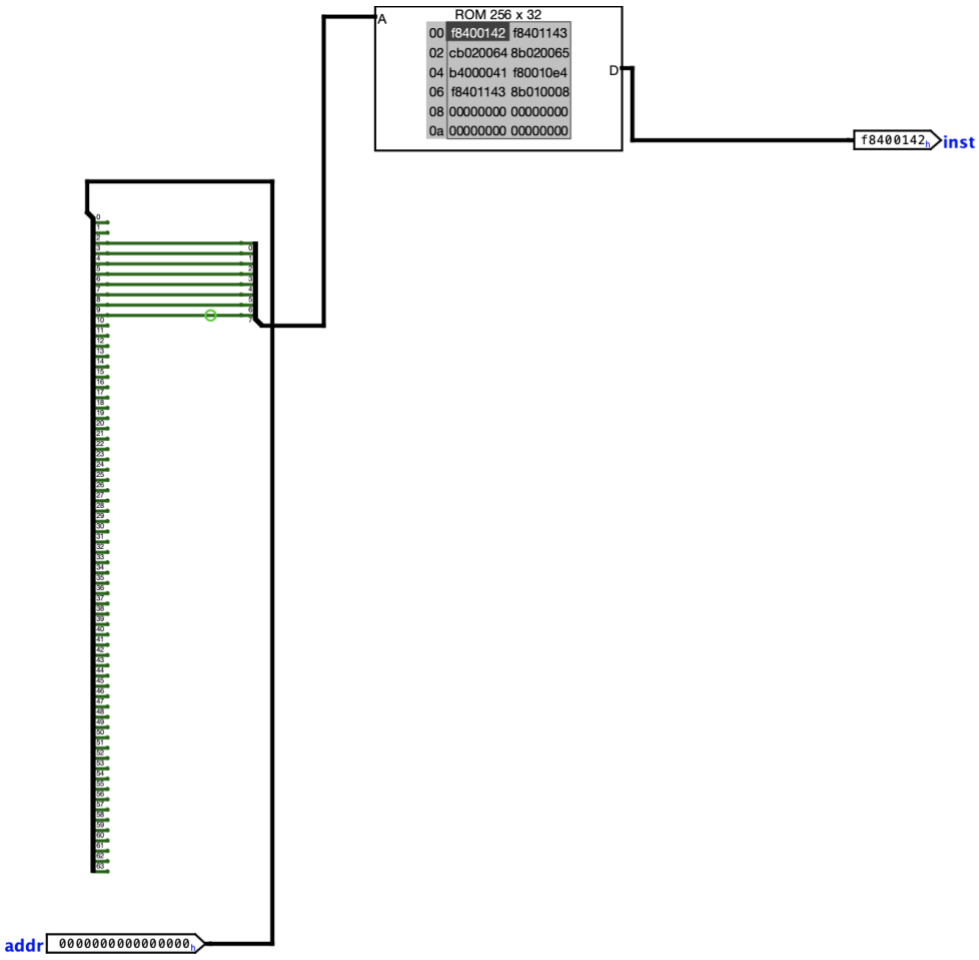
Program Counter (PC)

The Program Counter (PC) stores the address of the current instruction being executed. On each clock cycle, it updates to the next instruction address, either sequentially (PC + 4) or from a branch target. It serves as the starting point of instruction fetch in the data path.



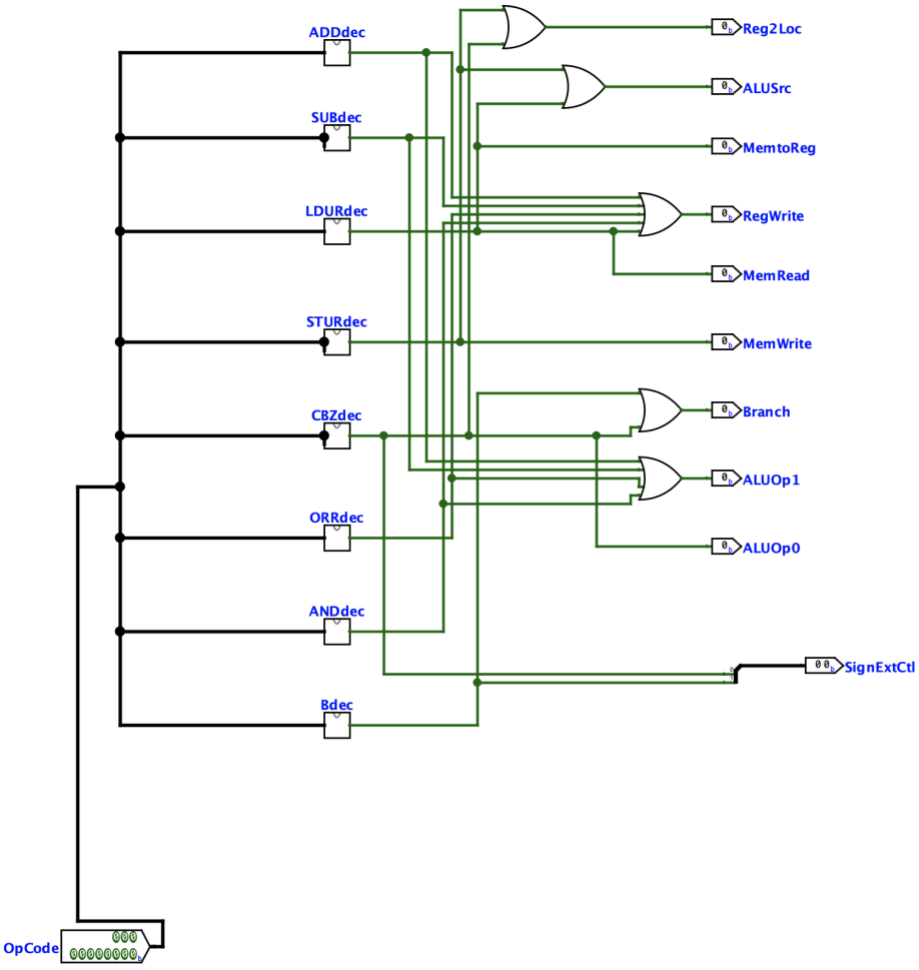
Instruction Memory

Instruction Memory stores the program instructions in binary (hex) format. It takes the address from the Program Counter and outputs the corresponding instruction. This instruction is then sent to the Control Unit and other Datapath components for execution.



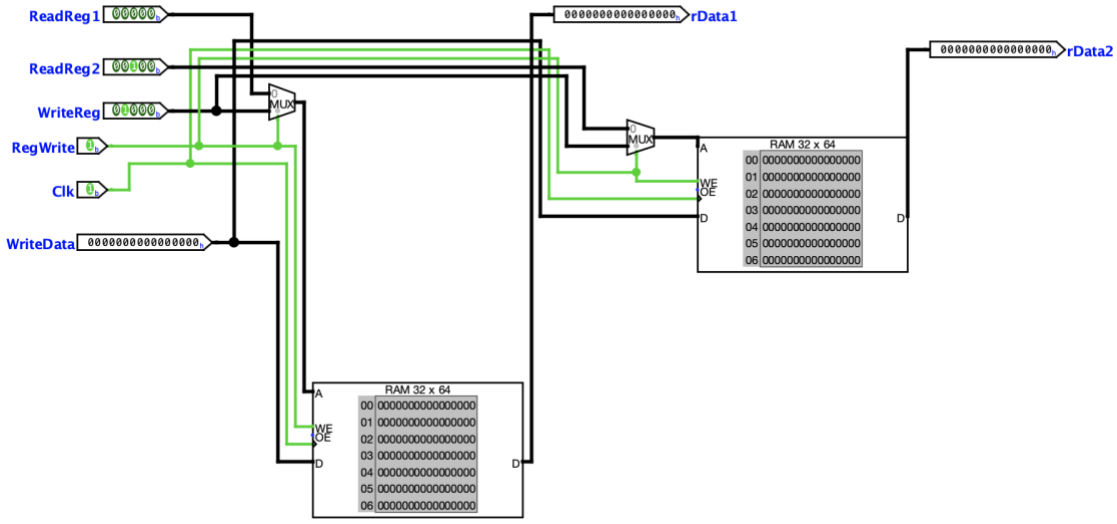
Control Unit

The Control Unit decodes the opcode field of the instruction and generates control signals for the datapath. These signals determine how data flows through the system, including register access, ALU operation selection, memory access, and branching behavior.


ADDdec
SUBdec
LDURdec
STURdec
CBZdec
ORRdec
ANDdec
Bdec
OpCode
Reg2Loc
ALUSrc
MemtoReg
RegWrite
MemRead
MemWrite
Branch
ALUOp1
ALUOp0
SignExtCtl

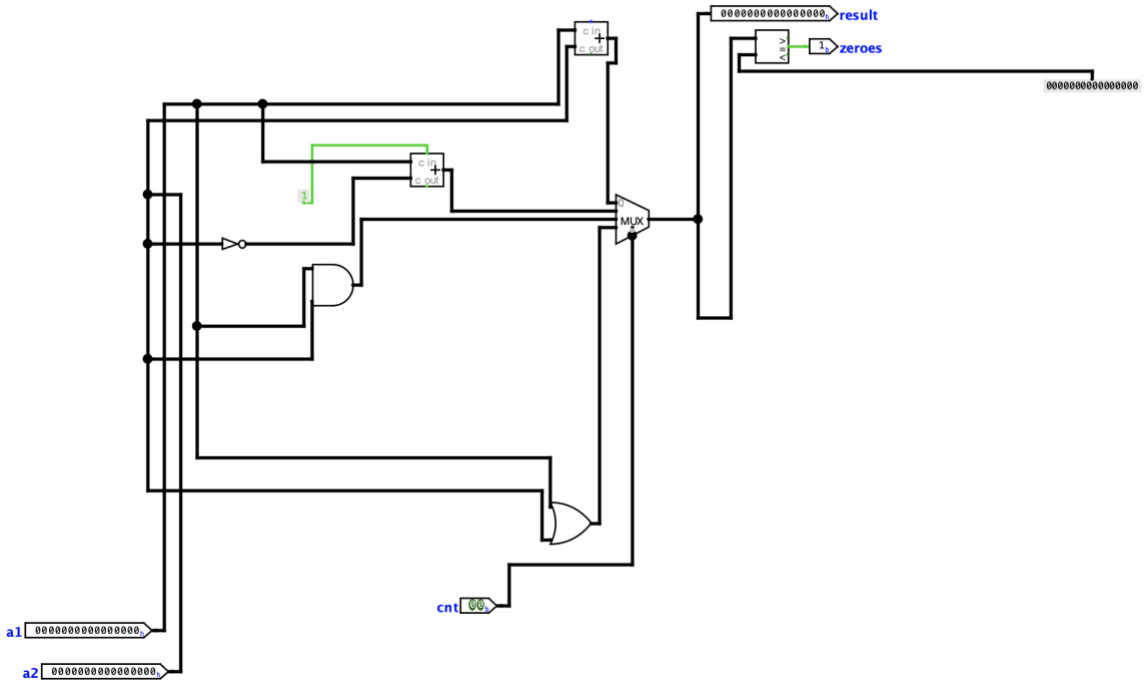
Register File

The Register File stores general-purpose registers used during execution. It supports two simultaneous reads, and one write per cycle. Based on control signals, it outputs operands to the ALU and writes results back into registers.



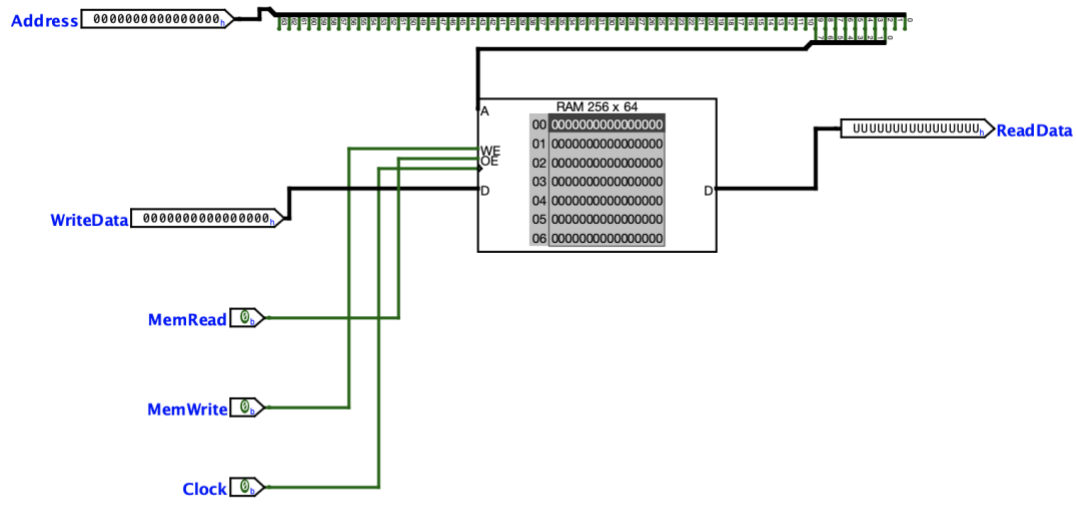
ALU

The ALU performs arithmetic and logical operations such as addition, subtraction, AND, and OR. It takes inputs from the Register File or immediate values and produces a result along with a zero-flag used for branch decisions.



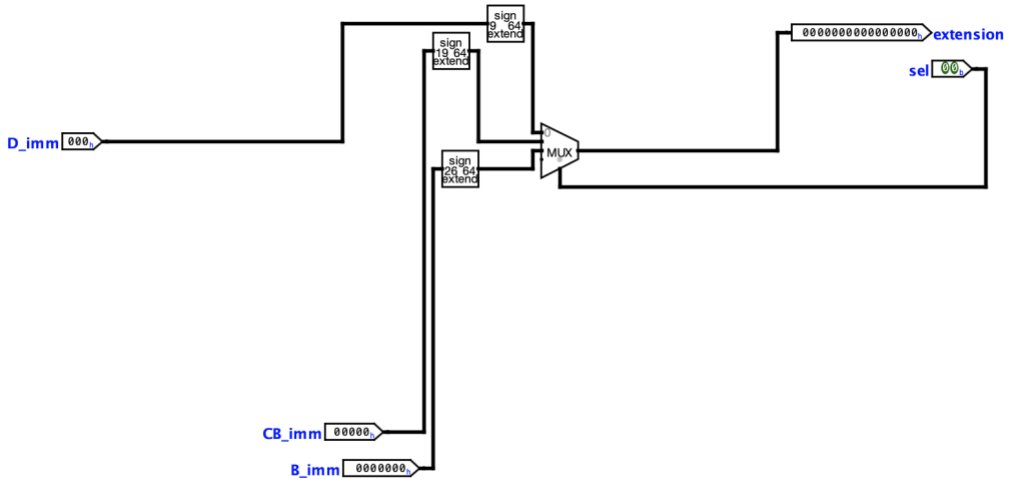
Data Memory

Data Memory is used for load and store instructions. It reads data from memory when executing load operations (LDUR) and writes data to memory during store operations (STUR), based on control signals.



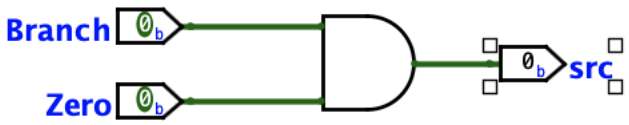
Sign Extend

The Sign Extend unit takes immediate values from the instruction and expands them to 64 bits while preserving the sign. Different instruction formats (D-type, CB-type, B-type) use different immediate sizes, so this unit selects and extends the correct field based on control signals. The extended value is then used by the ALU for address calculation or branch target computation.



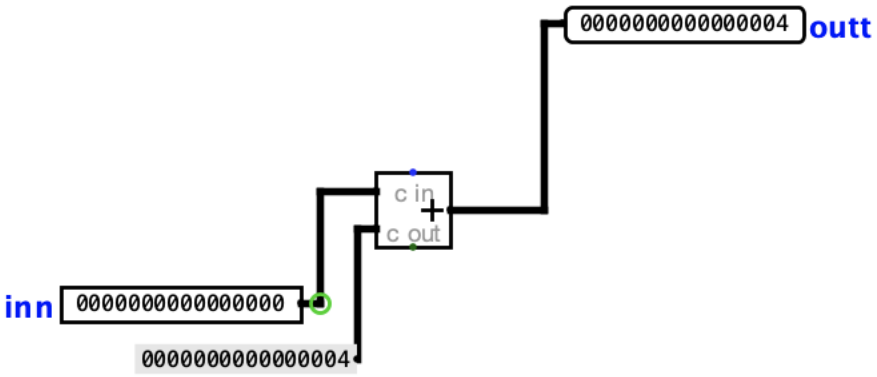
Branch Logic

The Branch Logic unit determines whether a branch instruction should be taken. It uses the Branch control signal from the Control Unit and the Zero flag from the ALU. If the branch condition is satisfied (e.g., CBZ and Zero = 1), it selects the branch target address; otherwise, execution continues sequentially. This unit controls the PC selection through multiplexers.



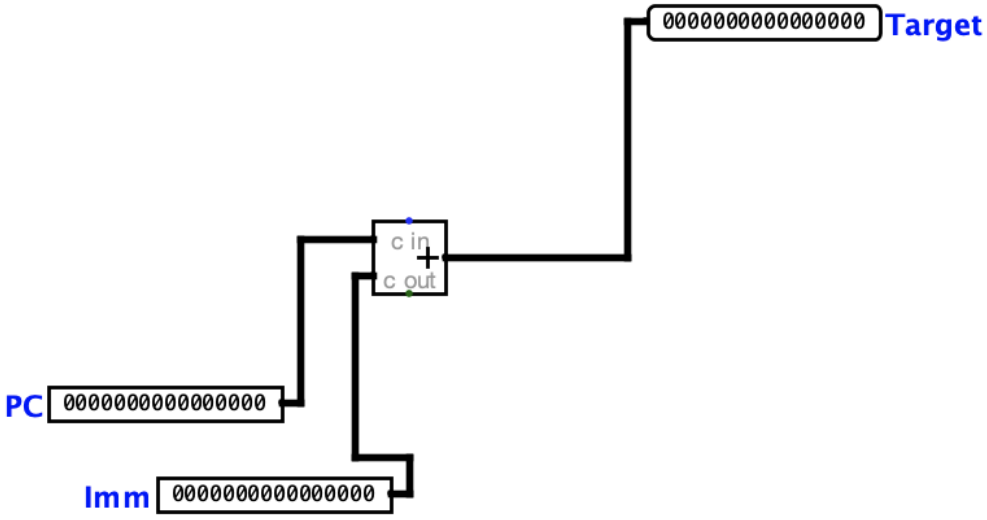
PC Adder

The PC Adder computes the next sequential instruction address by adding 4 to the current PC value. This ensures instructions are fetched in order unless a branch occurs.



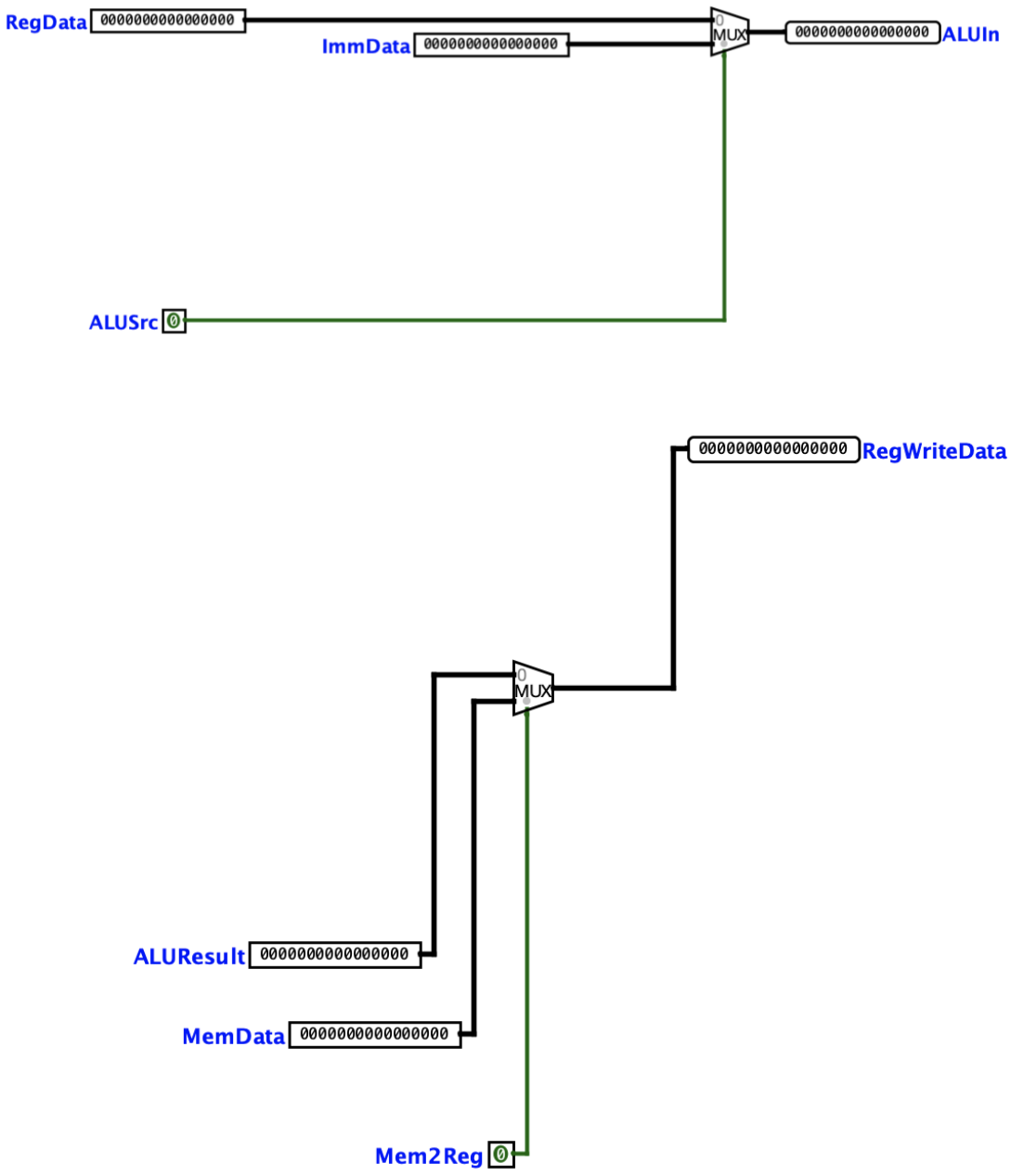
Branch Adder

The Branch Adder calculates the branch target address by adding the sign-extended immediate value to the current PC. This address is used when a branch instruction is taken.



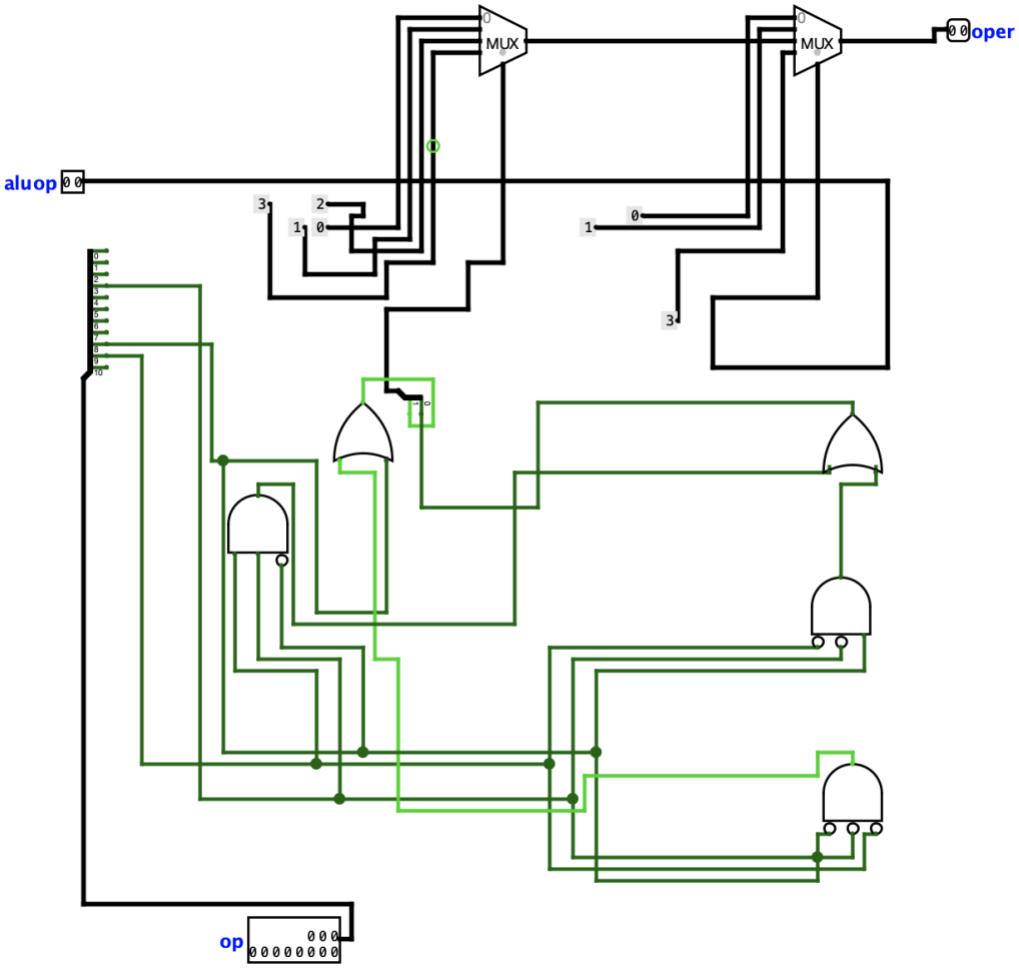
MUXes

Multiplexers are used throughout the datapath to select between multiple data sources based on control signals. Examples include selecting ALU inputs (register vs immediate), choosing write-back data (ALU result vs memory), and selecting the next PC value (sequential vs branch target).



ALU Control

The ALU Control unit determines the specific operation the ALU performs. It uses the ALUOp signals from the Control Unit along with instruction fields to generate the correct control signal for operations such as ADD, SUB, AND, and OR.



8. Conclusion

This project involved the design and implementation of a single cycle datapath processor using Logisim. Through this process, key components such as the Program Counter, Instruction Memory, Control Unit, Register File, ALU, Data Memory, and supporting modules were integrated into a complete working system. The datapath successfully executes different instruction types, including arithmetic, memory, and branch operations, within a single clock cycle.One of the main challenges of this project was ensuring correct communication between all components. Designing the Control Unit to generate accurate control signals based on instruction opcodes required careful attention to detail, as even small errors could cause incorrect behavior in the datapath. Additionally, debugging the connections between modules especially the branch logic and memory operations was time-consuming and required a thorough understanding of how data flows through the processor.Overall, this project provided valuable hands-on experience with processor design and reinforced concepts learned in class, such as instruction execution, control signal generation, and datapath organization. Despite its complexity, completing this project improved my understanding of how a CPU operates at the hardware level but some issues are still persistent in my design.